

Ideal ADE

Applying Human-Agent Market Findings to a Harness OS

Architecture / product study — what an ideal ADE *should* enforce

Inputs: *Agents Meet Humans* · AGENTS.md · DEC-A-010 / 013 / 014

Companion: *What ADE Could Do* (this monorepo)

Edition: 2026-07-23

Type: Architecture / product study
Audience: ADE maintainers, harness designers
Stance: Ideal ADE — what *should* exist if quality $\approx$ model $\times$ harness $\times$ human process
Not: Implementation backlog for one sprint (see companion paper)

Abstract

The market study says agent products fail when they treat fluency as truth, mutate without contracts, parallelize without locks, praise without sensors, and meter dollars that lie. An **ideal ADE** is not a better chat pane or an IDE fork. It is a **local harness OS**: every side effect consumes an authorized envelope; every parallel writer holds a lease; every Automate exit runs verify outside the generator; every spend number matches an invoice class; Continuity replaces paste thrash; humans escalate by risk, not by vibes.

This paper maps each market finding to an ideal ADE layer, names the product objects that enforce it, and ranks design principles for builders who already believe harness > chrome.

1. Ideal product definition

ADE (ideal) = Agent Development Environment = **local multi-host agent operating system**.

It is	It is not
Control plane for Suggest / Apply / Automate	A VS Code / Zed fork as identity
Authority over tools, paths, spend, verify	A prompt wrapper with pretty panels
Multi-agent with hierarchy + locks	Peer democracy of chatting agents
Host-agnostic brain (Desktop + optional editor eyes)	“Whatever editor Chrome we ship first”
Dogfoodable against itself	Demo-only Mission Control

Equation (locked):

Outcome quality $\approx$ model capability $\times$ harness discipline $\times$ human process

Ideal ADE owns the middle term completely and scaffolds the third.

2. Translation table — market finding → ideal ADE

Market finding	Ideal ADE response	Product object
Fluency $\neq$ truth	Separate generator from judge ; Automate requires verify	Verify ladder + Verifier slot
Blank retry / thrash	Terminalize failures; Fix&retry / Continue / handoff CTAs	Turn law + Continuity
Underspec before mutate	Contract before Apply (goal · AC · OOS · verify cmds)	Eng-goal + clarify chips (≤ 3)
Context dump	Skills on demand; thrift prompts; capsule resume	Skills catalog + handoff

Model roulette	Explicit model profiles + router; no silent swap	ade.model-profile/v1
Self-certify	Sensors outside generator; gold / sealed evals	VerifyRunner + gold harness
Sycophancy	Rules that forbid praise-as-pass; evidence-first research skill	Always-on rules + skills
Review habituation	Risk-tiered HITL; small diffs; override metrics	Autonomy dial + risk policy
Peer multi-agent tax	Planner ≠ Worker ≠ Verifier; claim-one; leases	Slot Orchestrator + leases
\$/outcome opacity	Honest SpendGuard; caps that halt; fertility later	Spend ledger + Trust UI
Verticals need envelopes	Coding = leases+verify; support = rollback; sales = draft/send; creative = brand kit	Packs / profiles per domain
Differentiation = production survival	Dogfood + gold races + Continuity under budget	Harness eval suite

3. Ideal layered architecture

Human process (guided) goal · clarify · approve risk · review verify · spend eye
Control plane (Desktop / CLI) Suggest Apply Automate · Effort · Trust · Capsules
Orchestrator slots · task claim · heartbeats · Isolate worktrees
Authority ToolEffect · leases · PLAN · rules · action envelopes
Turn runtime budgets · Continuity · skills activate · host tools
Sensors Verify G0-G5 · gold · spend reconcile · lease conflicts
Identity on disk .ade/{rules,skills,profiles,leases,tasks,goals,chat,...}
Optional eyes Editor host via ACP · browser · terminal – never the brain

Design rule: Eyes can change. Brain (authority + turn + verify + Continuity) must not.

4. Ideal principles (normative)

P1 — Side effects consume envelopes

No tool that mutates filesystem, shell, network, or money may run on raw model prose. Ideal envelope fields:

- effect class (read / write / exec / network / spend)
- path or resource set
- lease id (if write)
- autonomy mode at grant time
- plan phase / eng-goal id (if Act+)
- human override flag (if any)

Envelope is logged into the turn trace and eligible for Continuity.

P2 — Contract before Apply

Ideal gate: **no Act-class tools** until an active eng-goal (or equivalent contract) exists with:

- goal statement
- acceptance criteria
- out-of-scope
- verify commands (or pointer to verify ladder profile)

Clarify loop capped (≤ 3 questions). Suggest may explore without contract; Apply may not.

P3 — Hierarchy beats peer chat

Ideal slots:

Slot	Job	Must not
Planner (Suggest)	Intent, PLAN, task queue	Write production paths
Worker (Apply)	Claim one task; mutate under lease	Self-approve verify
Verifier	Run sensors; grade evidence	Generate the patch under test
Spend / Ops (optional)	Cap, reconcile, halt	Rewrite product code

No flat “agents discuss until consensus” as default.

P4 — One writer per path

Leases are truth. Worktrees isolate FS; leases stay on primary identity. Conflict UX must be actionable (wait / isolate / rotate), not a red badge with no next step.

P5 — Automate = Act + required verify

Ideal Automate cannot complete successfully without a verify pass (or explicit human waive recorded as override). Waive rate is a product metric.

P6 — Spend numbers tell invoice truth

Ideal SpendGuard:

- non-zero rates where provider bills
- reserve $\approx$ expected turn, not full-context fiction
- cache / billable fields when known
- session + daily hard stops
- Trust UI shows used / reserved / remaining in the same units as invoices

P7 — Continuity over paste

Ideal default: at budget pressure or long thread, emit capsule; host may run `next_safe_command`; resume thrift. Humans should not be trained to dump prior chats.

P8 — Profiles are first-class amplification

Ideal **model profile** (not just guidance packs):

- default autonomy ceiling
- tool mask
- effort floor
- spend ceiling
- slot eligibility

Router chooses profile by role $\times$ risk $\times$ budget — never silent mid-task swap without user-visible reason.

P9 — Skills on demand; rules always-on for safety

Ideal: thin always-on deny/invariant rules; fat procedures as activate-able skills. Catalog-first, not context dump.

P10 — Harness is the critical path; editor is optional eyes

Ideal roadmap never blocks SpendGuard / Orchestrator / envelopes on ACP completeness. Soft-shell editor later; hard fork last.

5. Ideal human process (scaffolded by product)

Market study: humans waste money on underspec, blank retry, dump, roulette, self-certify.

Ideal ADE scaffolds **anti-habits**:

Habit to kill	Scaffold
Underspec	Eng-goal form + ≤ 3 clarify before Apply
Blank retry	Terminalized failure + Fix&retry / Continue
Context dump	Continuity CTA + thrift resume
Model roulette	Profile picker + “why this model” chip
Self-certify	Verify required for Automate; evidence in Trust
Sycophancy	Rule: no “looks good” without sensor
Review rubber-stamp	Risk-tiered confirm; small leased diffs

Human remains accountable for goals and overrides. ADE makes the good path the short path.

6. Ideal vertical packs (same OS, different envelopes)

Coding-first ADE can still ship **packs** without becoming five products:

Pack	Envelope emphasis
money	Integer micros, caps, no float money
secrets	Path deny, no secret echo
infra	Destructive cmd HITL
support (future)	Rollback + transcript contract
sales (future)	Draft-only until human send
creative (future)	Brand kit bounds
regulated (future)	Supervised-only + audit export

Ideal ADE differentiates by **production survival + governance + metering**, not by claiming a vertical chat bot.

7. Ideal success metrics

Metric	Ideal direction
Redo-loop rate after Apply	↓
Verify pass rate on Automate completes	↑
Human override / waive rate	Visible; not hidden
Dual-writer lease violations	→ 0
Spend UI vs invoice delta	→ 0
Continuity resume success	↑
Gold harness regressions	0 on main
Time-to-contract before first mutate	↓

8. Ideal anti-goals

- Identity as “Cursor clone” or “Zed fork”
- Mission Control UI before slot truth
- Peer agent democracy as default
- Caps that show \$0 while tokens burn
- Automate that greenlights on model self-report
- Dumping entire chat into every turn
- Blocking harness on editor chrome

9. Ideal roadmap shape (not a sprint plan)

Foundation	Authority · turn law · leases · verify · Continuity
↓	
Honesty	SpendGuard truth · envelopes in trace
↓	
Contracts	Eng-goal gate · clarify · risk HITL
↓	
Slots	Planner ≠ Worker ≠ Verifier product truth
↓	
Amplification	Model profiles + router
↓	
Eyes	Optional ACP / editor soft shell
↓	
Enterprise	SSO / RBAC / Mission Control (only after slots)

10. Conclusion

An ideal ADE applies the market study by making **discipline productized**:

1. Envelopes for side effects
2. Contracts before Apply
3. Hierarchy + leases for multi-agent
4. Sensors outside the generator
5. Honest spend
6. Continuity instead of paste
7. Profiles that amplify models without roulette
8. Harness-first; editor optional

The companion paper maps this ideal onto **today's ADE** and what it can ship next.

References (internal)

- docs/research/Agents-Meet-Humans-Market-Study.md
- docs/research/ADE-Tokenomics-IO-Context-Usage.md (I/O · context · usage — **not** \$)
- docs/research/ADE-Agents-Harnesses-Token-Economics-Brief.md (harness / MAS / verify; \$ metering adjacent)
- AGENTS.md · DEC-A-010 · DEC-A-013 · DEC-A-014